

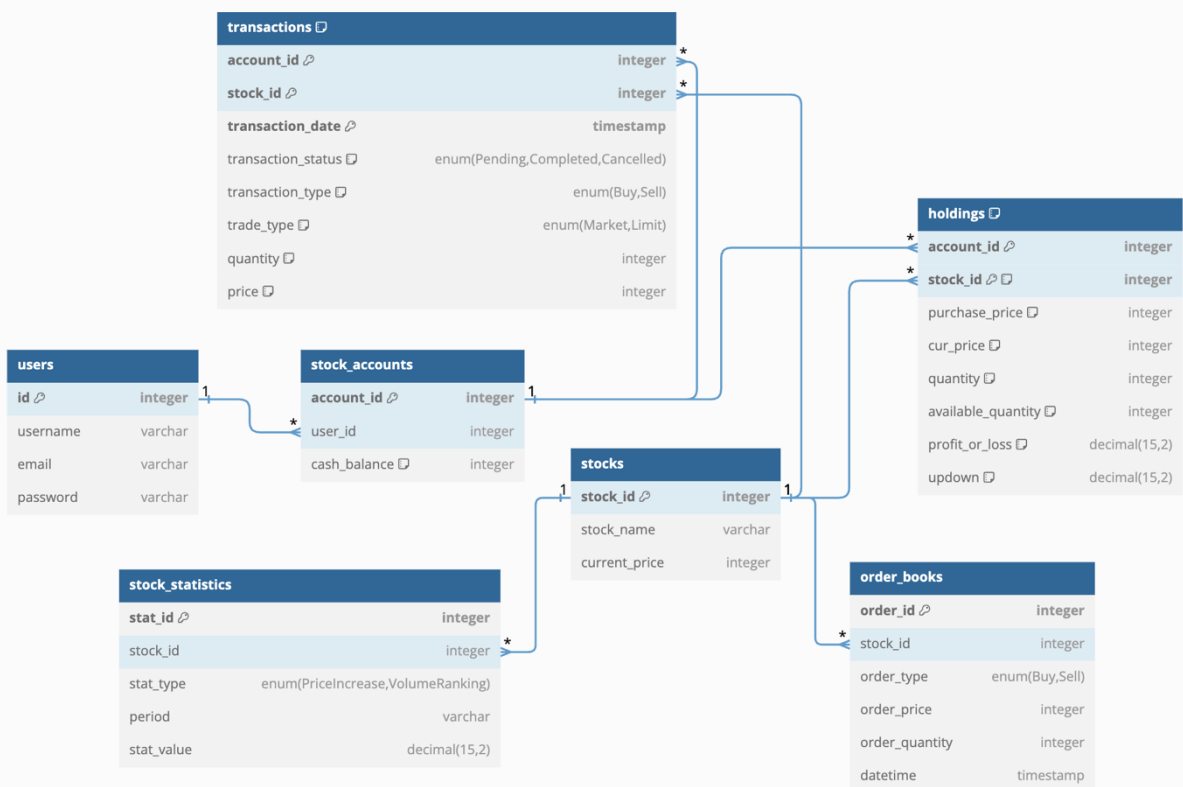
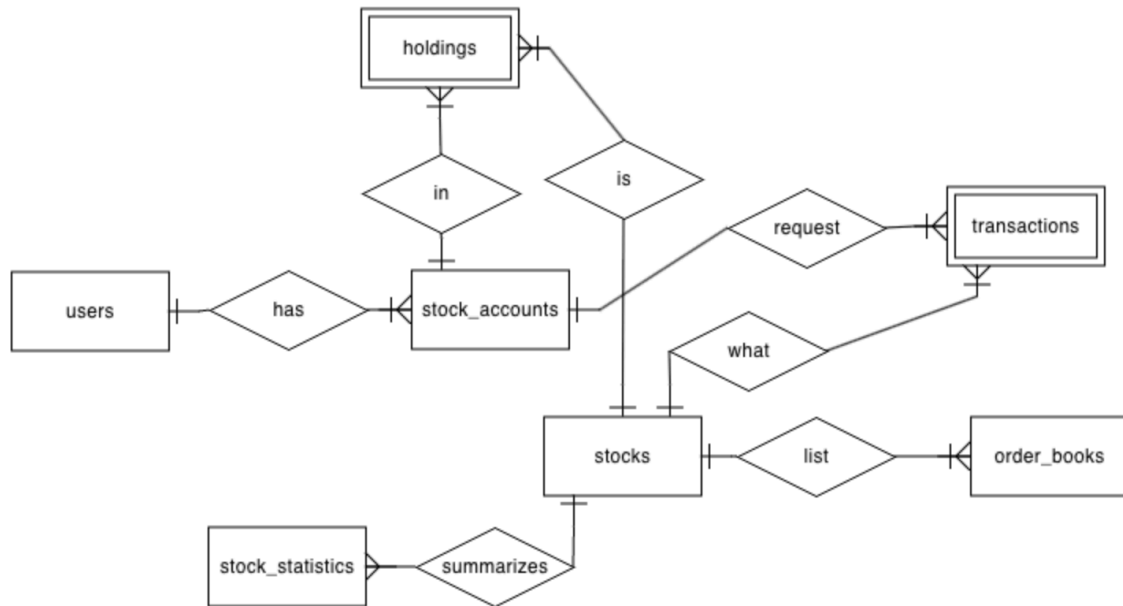
MileStone #1

E-R diagram, Relation schema

2019060546

주원웅

스키마 구조



1. users Entity

로그인에 필요한 Attribute 인 email, password 을 넣어주었고, 유저의 이름과 유저들을 식별하기 위한 id Attribute 를 넣어주었다. 사용자 ID는 AUTO_INCREMENT 속성을 통해 자동으로 증가하며, 각 사용자에게 고유한 ID를 부여한다.

Attribute:

id: Primary Key, 사용자 고유 식별자.

username: 사용자 이름.

email: 이메일 주소, Unique로 설정.

password: 사용자의 암호화된 비밀번호.

2. stock_accounts Entity

유저가 가지고있는 계좌이다. 계좌 번호인 Account_id 와 users entity의 id 를 foreign key 로 사용하는 user id attribute 를 사용한다. 주식 거래를 위한 사용자의 현금 잔액 case_balance attribute 를 사용한다.

User_id Attribute 를 users entity 의 id 와 many to one 와 관계로 사용하는데 그 이유는 한 사람이 여러개의 계좌를 가질 수 있기 때문이다.

Attribute:

account_id: Primary Key, 각 계좌의 고유 식별자.

user_id: Foreign Key, 계좌 소유자의 ID, users 테이블의 id를 참조.

cash_balance: 계좌에 보유된 현금 잔액

3. stocks Entity

주식을 나타내는 Entity 이다. 주식 ID를 통해 각 주식이 식별되며, 주식명과 현재 가격을 나타낸다.

Attribute:

stock_id: Primary Key, 주식 고유 식별자.

stock_name: 주식명(회사명 등).

current_price: 현재 거래 가격.

4. holdings Entity

사용자의 계좌(stock_accounts)에 보유한 주식을 표현하는 Entity 이다. 사용자가 보유 중인 주식을 나타낸다. Primary key 로 주식계좌, 주식을 사용하는데 그 이유는 주식계좌와 주식이 결정되고 어떤 매입 가격이 존재하면 어떤 주식 계좌 안에 해당 주식을 구매하였다고 판단하여 이 세가지를 primary key 로 사용했다. 각 주식도 다 다른 가격으로 구입할 수 있기 때문이다.

주식 계좌와 many to one 관계를 이루는 이유는 한 계좌 안에 여러 개의 holding 중인 주식이 존재할 수 있기 때문이다,

주식과 many to one 관계를 이루는 이유는 동일한 주식을 여러번 구매할 수 있기 때문이다. 동일한 주식을 가격이 다르게 여러번 구매가 가능한 것을 구현하기 위해 이런 관계를 구성했다.

Attribute:

account_id: Primary Key, Foreign Key, 계좌 ID, stock_accounts 테이블을 참조.

stock_id: Primary Key, Foreign Key, 주식 ID, stocks 테이블을 참조.

purchase_price: Primary Key, 사용자가 해당 주식을 매수한 가격.

cur_price: 주식의 현재 거래 가격 (stocks의 current_price와 동기화 해야한다).

quantity: 보유 중인 주식 수량.

available_quantity: 매도 가능한 주식 수량.

profit_or_loss: 평가 손익, (현재가 - 매수가) * 보유 수량으로 계산.

updown: 등락률, ((현재가 - 매수가) / 매수가) * 100으로 계산.

5. transactions Entity

사용자의 주식 거래 내역을 기록하는 테이블이다. 거래 상태, 거래 유형(매수/매도), 시장가/지정가 구분, 거래 가격 및 수량 의 정보를 나타낸다.

주식 계좌, 주식 Entity 와 many to one 관계를 이룬다. 주식에 대해 여러 transaction이 존재할 수 있고, 한 계좌에서도 여러 transaction 이 존재할 수 있기 때문이다.

Primary key 로 account_id, stock_id, transaction_date 를 사용하는데, 동일한 계좌에서 동일한 주식에 대해 시간별로 거래가 존재할 수 있기 때문이다.

Attribute:

account_id: Primary Key, Foreign Key, 계좌 ID, stock_accounts 테이블을 참조.

stock_id: Primary Key, Foreign Key, 주식 ID, stocks 테이블을 참조.

transaction_date: Primary Key, 거래가 발생한 날짜 및 시간.

transaction_status: 거래 상태 (대기 중, 완료, 취소).

transaction_type: 거래 유형 (매수, 매도).

trade_type: 거래 방식 (시장가, 지정가).

quantity: 거래 수량.

price: 거래 가격.

6. order_books Entity

주식 거래에 대한 주문 정보를 관리하고 5단계 호가창을 표현할 테이블이다. 매수/매도 주문 내역과 가격 및 수량을 기록하며, 주문 ID를 통해 고유하게 식별한다.

주문 ID 를 통해 고유하게 식별돼야 하는 이유는 5단계 호가창에 존재하는 가격들에 대해 동일한 수량을 보여주지만, 가장 먼저 등록된 주문 먼저 처리해야 하기 때문에 따로 primary key 에 대한 attribute를 추가해주었다. Datetime 을 통해 식별해도 되지만, 동일한 시간의 경우 식별할 방법이 없기 때문이다.

주식과 many to one 관계를 이루는데, 그 이유는 한 주식에 대해서 여러 주문이 존재할 수 있기 때문이다.

Attribute:

order_id: Primary Key, 각 주문의 고유 식별자.

stock_id: Foreign Key, 주식 ID, stocks 테이블을 참조.

order_type: 주문 유형 (매수, 매도).

order_price: 주문 가격.

order_quantity: 주문 수량.

datetime: 주문이 생성된 날짜와 시간.

7. stock_statistics Entity

주식 통계 정보를 관리하는 테이블로, 주식의 가격 변동률 또는 거래량 순위와 같은 통계를 기록한다. 특정 주식에 대한 통계 정보를 저장하고, stat_type 필드를 통해 통계 유형을 구분한다.

주식과 many to one 관계를 이루는데, 그 이유는 한 주식에 대해서 기간별, 종류 별로 여러 통계가 있을 수 있기 때문이다.

Attribute:

stock_id: Primary Key, Foreign Key, 주식 ID, stocks 테이블을 참조.

stat_type: Primary Key, 통계 유형 (가격 상승률, 거래량 순위).

period: Primary Key, 통계가 적용되는 기간.

stat_value: 통계 값 (예: 가격 상승률 또는 거래량 순위).